

## Găsiți ordinea

- **Limită de timp:** 10 minute
- **Mediu:** un GPU ( $\approx 16$  GB VRAM), fără internet
- **Dimensiunea soluției:** `solution.ipynb`  $\leq 1$  MB
- **Spațiu de stocare:** 5 GB

## Problemă

Primiți dialoguri în limba engleză vorbită între doi participanți, *Vorbitorul A* și *Vorbitorul B*. Fiecare dialog este segmentat în replici, fiecare replică incluzând vorbirea unui singur vorbitor. Fiecare replică este stocată ca un fișier audio `.wav` separat, astfel încât un dialog complet este reprezentat printr-un set de fișiere `.wav`, câte unul pentru fiecare replică.

Din păcate, replicile au fost amestecate aleatoriu, astfel încât conversația nu mai are sens. În numele de fișier `chunk_{k}.wav`,  $k$  se referă la al  $k$ -lea segment din setul amestecat, nu la a  $k$ -a replică din dialogul original.

**!! Sarcina dumneavoastră este să reconstruiți ordinea cronologică originală a conversației.**



## Dataset

Fiecare dialog conține fișiere audio  $n$  denumite `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Segmentele sunt replici individuale. Numele fișierelor corespund doar ordinii amestecate. Acestea nu indică poziția unui segment în conversația originală. Fiecare dialog are 7–20 segmente, mono, 44.1 kHz (puteți reeșantiona).

**`prefix.json` conține indicii din numele fișierelor pentru primele două segmente din fiecare dialog.** Acest lucru identifică începutul real al dialogului și elimină ambiguitatea dintre parcurgerea conversației înainte sau înapoi.

De exemplu: 11: [7, 12] înseamnă că prima și a doua replică din dialogul 11 sunt `chunk_7.wav` și, respectiv, `chunk_12.wav`.

### Ce primiți

Primiți **două directoare cu format identic:**

| Director                          | Dialoguri | <code>answers.json</code> ? | Folosiți-l pentru a                             |
|-----------------------------------|-----------|-----------------------------|-------------------------------------------------|
| <code>dataset/train/</code>       | 1,288     | ✓ inclus                    | antrena / ajusta fin modelul dumneavoastră      |
| <code>dataset/test_public/</code> | 100       | ✓ inclus                    | rula pipeline-ul și calcula local propriul scor |

În timpul evaluării, directorul dumneavoastră `dataset/test_public/` este înlocuit în mod transparent cu un `hidden evaluation set` (`test_leaderboard_a` pentru clasamentul public și `test_leaderboard_b` pentru clasamentul final) — acestea au aceeași dimensiune și același format ca `dataset/test_public/`, dar fără `answers.json`.

Notebook-ul dumneavoastră este executat din nou pe acele date, iar fișierul `answers.json` pe care îl produce este utilizat pentru calcularea scorului. Dialogurile de test păstrate separat provin din aceeași distribuție ca `train`, astfel încât scorul dumneavoastră local `test_public` reprezintă o estimare fidelă.

## Structura directoarelor

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

## Ieșire

Pentru fiecare dialog, determinați ordinea cronologică originală a segmentelor sale audio. Predicția dumneavoastră trebuie să fie o permutare  $P$  a lui  $\{0, 1, \dots, n-1\}$ , unde  $P[i]$  este poziția cronologică prezisă pentru `chunk_i.wav` ( $0 = \text{primul}$ ).

Fișierul dumneavoastră de ieșire `answers.json` trebuie să asocieze fiecărui ID de dialog permutarea prezisă:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

## Exemplu

Un dialog are 3 segmente amestecate `chunk_0`, `chunk_1`, `chunk_2`:

| segment amestecat | conținut rostit                                     | poziție reală (rang) |
|-------------------|-----------------------------------------------------|----------------------|
| chunk_0.wav       | "No worries — I'll send you the notes afterwards."  | 2 (ultimul)          |
| chunk_1.wav       | "Hey, are you coming to the three o'clock meeting?" | 0 (primul)           |
| chunk_2.wav       | "I can't — I've got a dentist appointment then."    | 1                    |

Ordinea reală este **chunk\_1** → **chunk\_2** → **chunk\_0**, deci  $P = [2, 0, 1]$ , iar `prefix.json` conține  $[1, 2]$ .

**⚠ P trebuie să fie o permutare validă:** lungime  $n$ , indexată de la 0, fiecare valoare apărând exact o dată. Valorile duplicate, valorile lipsă sau elementele din afara intervalului (de exemplu, indexate de la 1) produc un scor de 0 pentru dialogul respectiv, la fel ca un dialog care lipsește din fișier. Un fișier cu format incorect sau care nu este JSON este respins.

## Evaluare

Evaluarea pentru această sarcină utilizează **acuratețea ordonării pe perechi**. Aceasta verifică fiecare pereche de segmente și întreabă: *care dintre cele două ar trebui să apară primul?* O pereche este corectă dacă predicția dumneavoastră oferă același răspuns ca ordinea de referință. Pentru un dialog cu  $n$  segmente există

$$M = n(n - 1)/2$$

perechi; fie  $I$  numărul de inversiuni — perechi ordonate diferit față de ordinea de referință:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

**i Scorul final este media scorurilor per dialog pentru toate dialogurile din subset.**

## Modele permise

Puteți folosi numai următoarele modele preantrenate pentru a rezolva această sarcină, atât în timpul antrenării, cât și al evaluării. Toate aceste modele sunt deja descărcate și disponibile în mediu. Puteți vedea exemple privind utilizarea lor în notebook-ul `baseline solution.ipynb`. Rețineți că nu puteți folosi niciun alt model, iar programul dumneavoastră nu are acces la internet.

- **Reprezentări ale vorbirii: wav2vec 2.0.** Encoderul **Whisper** poate fi, de asemenea, utilizat ca extractor de caracteristici. [Fișa modelului wav2vec](#)
- **Recunoaștere automată a vorbirii (ASR): OpenAI Whisper** (orice dimensiune). [Fișa modelului Whisper](#)

- **Model lingvistic: Qwen2.5-0.5B**, care poate fi utilizat fie zero-shot, fie ajustat fin pe subsetul `train` furnizat. [Fișa modelului Qwen](#) Rețineți că limita de 10 minute trebuie să includă orice antrenare sau ajustare fină pe care o efectuați în timpul evaluării, precum și inferența pe setul de evaluare.

## Cum să trimiteți soluția

- Deschideți `solution.ipynb` și rulați toate celulele. Confirmați că acesta scrie `answers.json` în directorul de lucru, cu câte o permutare pentru fiecare dialog din `dataset/test_public/` (100 dialoguri). În timpul evaluării, notebook-ul este rulat din nou pe setul de test ascuns, iar fișierul `answers.json` pe care îl produce acolo este evaluat.
- Îmbunătățiți soluția dacă doriți — sau nu; baseline-ul validează singur pipeline-ul.
- Deschideți fila Git din bara laterală stângă a JupyterLab.
- Aplicați **Stage** pentru `solution.ipynb` (pictograma + de lângă acesta).
- Introduceți un mesaj de commit și faceți clic pe **Commit**.
- Faceți clic pe pictograma nor cu săgeată în sus pentru a efectua push.
- Reveniți la această pagină a concursului și faceți clic pe **Submit**.

Trimiteți exact un fișier, denumit `solution.ipynb`.